BZAN 554:
Deep Learning for Business Applications

Training Deep Neural Networks

Michel Ballings

Outline

- Embeddings
- Backpropagation
- Faster optimizers
- Global, constant learning rate
  - Momentum
  - Nestorov's Accelerated Gradient
- Global, adaptive learning rate
  - Learning Rate Scheduling
- Local, adaptive learning rate
  - Adaptive Gradient (AdaGrad)
  - RMSprop
  - Adaptive Moment Estimation (ADAM)
- Vanishing/Exploding Gradient
  - Problem statement
  - Solution 1: Weight Initialization
  - Solution 2: Nonsaturating Activation Functions
  - Solution 3: Batch Normalization
  - Solution 4: Gradient Clipping
- Avoiding Overfitting with Regularization
  - Early Stopping
  - L-1 and L-2 Regularization
  - Dropout
  - Data Augmentation
- Depth vs Width

Embeddings

- An embedding is simply a dense layer with a linear activation function, but instead of using matrix multiplication requiring us to feed in a dummy vector, an embedding applies a fast lookup allowing us to feed in the integer corresponding to the category.
- Why do we call it an embedding and not a dense linear layer? The term "embedding" comes from mathematics and representation learning, and "dense layer" comes from engineering.
- **One-hot encoding example**: Consider a categorical variable with 20 unique categories. Only one category can be 1, all other categories are 0. Consider the first hidden layer to have 2 units.

```
#Standard way: feed in dummy vector with 20 columns:
inputs = tf.keras.layers.Input(shape=(20,))
hidden = tf.keras.layers.Dense(2)(inputs)
#...

#Embedding: feed in 1 integer indicating category that is 1
inputs = tf.keras.layers.Input(shape=(1,))
embedding = tf.keras.layers.Embedding(input_dim=20,
                                      output_dim=2)(inputs)
#...
```

Embeddings

- **K-hot encoding example**: Consider a categorical variable with 20 unique categories. K (say 4) categories can be 1, all other categories are 0. Consider the first hidden layer to have 2 units.

```
k = 4
#Standard way: feed in dummy vector with 20 columns:
inputs = tf.keras.layers.Input(shape=(20,))
hidden = tf.keras.layers.Dense(2)(inputs)
#...

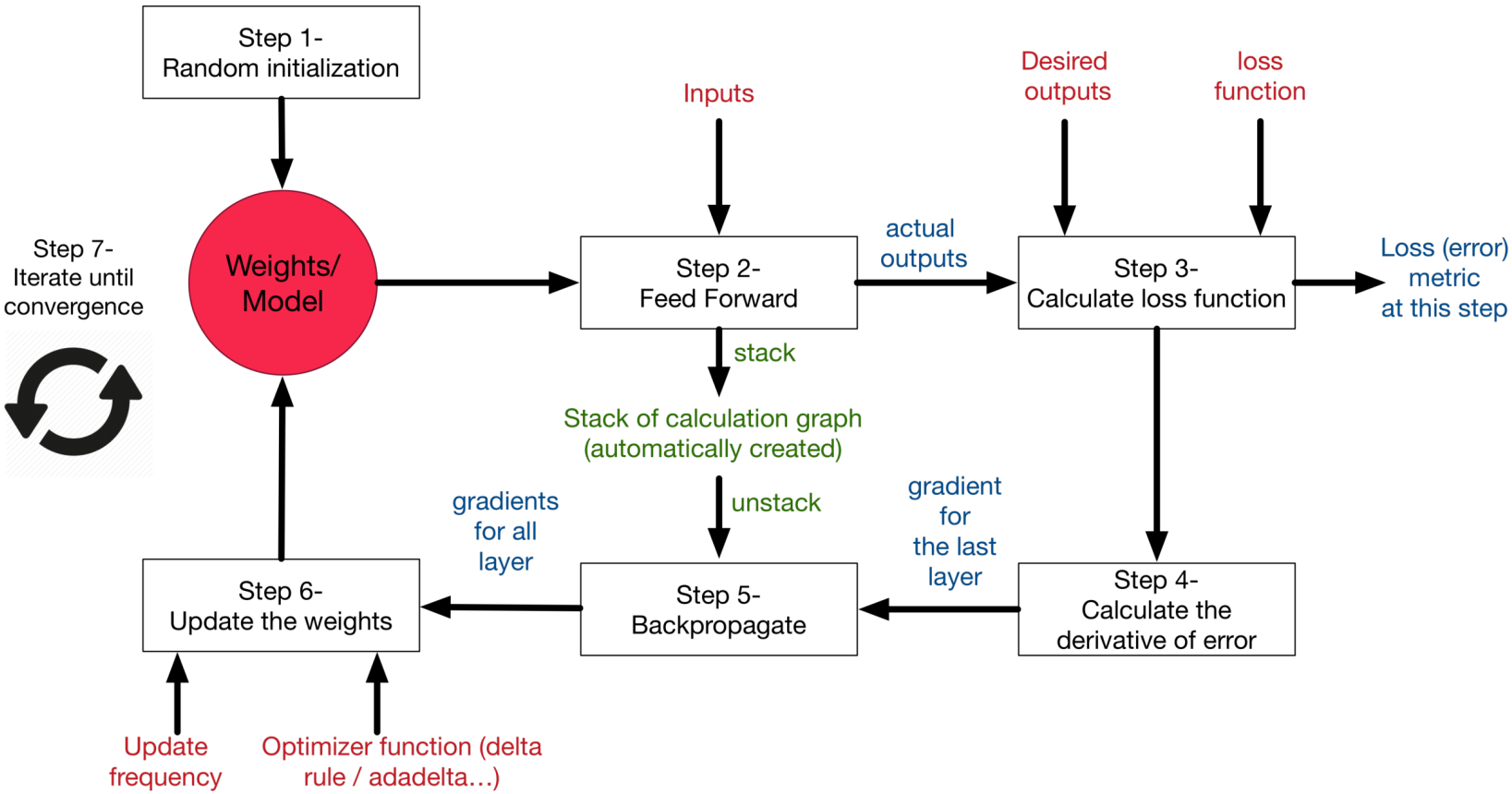
#Embedding: feed in k integers indicating categories that are 1
inputs = tf.keras.layers.Input(shape=(k,))
embedding = tf.keras.layers.Embedding(input_dim=20,
                                      output_dim=2)(inputs)
#...
```

- Embeddings cannot be used if the equivalent linear dense layer takes in values other than 0 or 1 (e.g., counts of words in text analysis).
- Situations with a dynamic k can be solved for with padding.
- For embeddings: make sure to integer-encode the input with consecutive integers. The input_dim is then max(categoricalvariable) + 1 (to account for zero-based counting).
- See **embedding.py**

Outline

- Embeddings
- Backpropagation
- Faster optimizers
- Global, constant learning rate
  - Momentum
  - Nestorov's Accelerated Gradient
- Global, adaptive learning rate
  - Learning Rate Scheduling
- Local, adaptive learning rate
  - Adaptive Gradient (AdaGrad)
  - RMSprop
  - Adaptive Moment Estimation (ADAM)
- Vanishing/Exploding Gradient
  - Problem statement
  - Solution 1: Weight Initialization
  - Solution 2: Nonsaturating Activation Functions
  - Solution 3: Batch Normalization
  - Solution 4: Gradient Clipping
- Avoiding Overfitting with Regularization
  - Early Stopping
  - L-1 and L-2 Regularization
  - Dropout
  - Data Augmentation
- Depth vs Width

Backpropagation



Backpropagation

- Backpropagation is very important for training deep networks as it enables us to recycle deeper calculations in the shallower calculations. This increases efficiency substantially, thereby making deep learning possible.
- See **backprop.pdf** for a fully worked example.

Outline

- Embeddings
- Backpropagation
- Faster optimizers
- Global, constant learning rate
  - Momentum
  - Nestorov's Accelerated Gradient
- Global, adaptive learning rate
  - Learning Rate Scheduling
- Local, adaptive learning rate
  - Adaptive Gradient (AdaGrad)
  - RMSprop
  - Adaptive Moment Estimation (ADAM)
- Vanishing/Exploding Gradient
  - Problem statement
  - Solution 1: Weight Initialization
  - Solution 2: Nonsaturating Activation Functions
  - Solution 3: Batch Normalization
  - Solution 4: Gradient Clipping
- Avoiding Overfitting with Regularization
  - Early Stopping
  - L-1 and L-2 Regularization
  - Dropout
  - Data Augmentation
- Depth vs Width

Faster variations of Gradient Descent

There are other variations of Gradient Descent incorporating ideas to allow faster convergence to the optimum. The most popular algorithms are:

- A global (same for all parameters) and constant (across learning updates) learning rate
  - SGD with Momentum
  - Nestorov's Accelerated Gradient (NAG)
- A global (same for all parameters) and adaptive (across learning updates) learning rate
  - Learning rate scheduling
- A local (per-parameter) adaptive (across learning updates) learning rate
  - Adaptive gradient (AdaGrad)
  - RMSprop
  - Adam

Outline

- Embeddings
- Backpropagation
- Faster optimizers
- Global, constant learning rate
  - Momentum
  - Nestorov's Accelerated Gradient
- Global, adaptive learning rate
  - Learning Rate Scheduling
- Local, adaptive learning rate
  - Adaptive Gradient (AdaGrad)
  - RMSprop
  - Adaptive Moment Estimation (ADAM)
- Vanishing/Exploding Gradient
  - Problem statement
  - Solution 1: Weight Initialization
  - Solution 2: Nonsaturating Activation Functions
  - Solution 3: Batch Normalization
  - Solution 4: Gradient Clipping
- Avoiding Overfitting with Regularization
  - Early Stopping
  - L-1 and L-2 Regularization
  - Dropout
  - Data Augmentation
- Depth vs Width

Outline

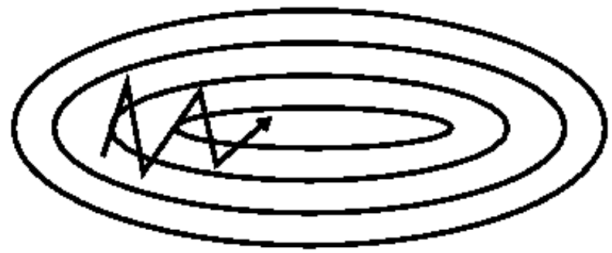
- Embeddings
- Backpropagation
- Faster optimizers
- Global, constant learning rate
  - Momentum
  - Nestorov's Accelerated Gradient
- Global, adaptive learning rate
  - Learning Rate Scheduling
- Local, adaptive learning rate
  - Adaptive Gradient (AdaGrad)
  - RMSprop
  - Adaptive Moment Estimation (ADAM)
- Vanishing/Exploding Gradient
  - Problem statement
  - Solution 1: Weight Initialization
  - Solution 2: Nonsaturating Activation Functions
  - Solution 3: Batch Normalization
  - Solution 4: Gradient Clipping
- Avoiding Overfitting with Regularization
  - Early Stopping
  - L-1 and L-2 Regularization
  - Dropout
  - Data Augmentation
- Depth vs Width

Momentum

SGD has trouble navigating plateaus. A plateau is common when variables have different scales (for example the elongated bowl in the figure happens because the feature representing the horizontal axis has much larger values than the one representing the y-axis)



(a) SGD without momentum



(b) SGD with momentum

Picture a cost function that goes down steeply, then almost plateaus with a near-horizontal downward slope and then goes down steeply again. On the steep regions SGD will go fast, and on the horizontal region SGD will go very slowly. When momentum is added the plateau is traversed much faster.

Momentum

This technique (`https://arxiv.org/pdf/1609.04747`) modifies the standard SGD by introducing velocity ν, and momentum γ which tries to control the velocity and prevents overshooting valleys while allowing faster descent as well as giving Gradient Descent a short-term memory.

$$\begin{aligned} \nu_t &= \gamma \nu_{t-1} + (1-\gamma)\nabla_\theta J(\theta) \\ \theta &= \theta - \eta \nu_t \end{aligned} \tag{1}$$

The momentum term γ is usually set to 0.9. Summarizing, the momentum algorithm replaces the gradient with a weighted moving average of the gradient and uses that to update the weights. It therefore memorizes how fast we were descending before and uses that speed to traverse plateaus. A way to think about this is a ball that rolls down a curve. If it hits a perfectly horizontal plateau it will continue rolling for a while until it has lost all its velocity. In plain vanilla SGD it would have stopped immediately when hitting the plateau.

Tensorflow Implementation of Momentum

Implementing Momentum optimization in Tensorflow:

```
tf.keras.optimizers.SGD(learning_rate=0.1, momentum=0.9)
```

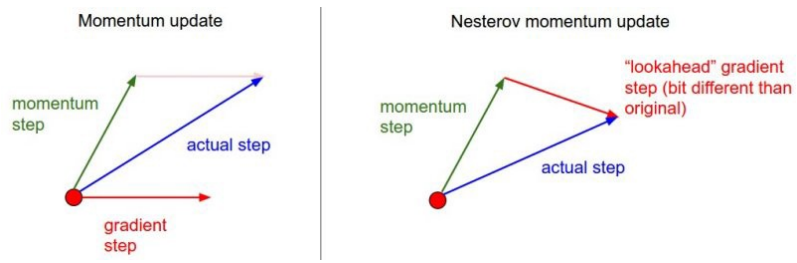
Outline

- Embeddings
- Backpropagation
- Faster optimizers
- Global, constant learning rate
  - Momentum
  - Nestorov's Accelerated Gradient
- Global, adaptive learning rate
  - Learning Rate Scheduling
- Local, adaptive learning rate
  - Adaptive Gradient (AdaGrad)
  - RMSprop
  - Adaptive Moment Estimation (ADAM)
- Vanishing/Exploding Gradient
  - Problem statement
  - Solution 1: Weight Initialization
  - Solution 2: Nonsaturating Activation Functions
  - Solution 3: Batch Normalization
  - Solution 4: Gradient Clipping
- Avoiding Overfitting with Regularization
  - Early Stopping
  - L-1 and L-2 Regularization
  - Dropout
  - Data Augmentation
- Depth vs Width

Nestorov's Accelerated Gradient

- We would like our ball to look ahead and slow down if the loss is going to go up. Nesterov's Accelerated Gradient is a small variation of momentum.
- Instead of calculating the gradient at the current position θ, we calculate the gradient at the approximate position that we will be in. Which information do we have (no additional computations) that we can add to θ before we compute the gradient? We know for sure that we are going to subtract $\eta\gamma\nu_{t-1}$ from θ!
- From a physical perspective, it makes sense to make judgments about our position based on the position that we know that we are going to be in a short while.

$$\begin{aligned} \nu_t &= \gamma\nu_{t-1} + (1-\gamma)\nabla_\theta J(\theta - \eta\gamma\nu_{t-1}) \\ \theta &= \theta - \eta\nu_t \end{aligned} \tag{2}$$



Tensorflow Implementation of NAG

- This strategy works because in general the momentum vector will be pointing towards the optimum, so it will be slightly more accurate to compute the gradient at a point closer to the optimum than at the original position.
- Nestorov's Accelerated Gradient will almost always speed up training compared to regular Momentum optimization.

```
tf.keras.optimizers.SGD(learning_rate=0.1, momentum=0.9,
nesterov=True)
```

https://towardsdatascience.com/stochastic-gradient-descent-with-momentum-a84097641a5d

Outline

- Embeddings
- Backpropagation
- Faster optimizers
- Global, constant learning rate
  - Momentum
  - Nestorov's Accelerated Gradient
- Global, adaptive learning rate
  - Learning Rate Scheduling
- Local, adaptive learning rate
  - Adaptive Gradient (AdaGrad)
  - RMSprop
  - Adaptive Moment Estimation (ADAM)
- Vanishing/Exploding Gradient
  - Problem statement
  - Solution 1: Weight Initialization
  - Solution 2: Nonsaturating Activation Functions
  - Solution 3: Batch Normalization
  - Solution 4: Gradient Clipping
- Avoiding Overfitting with Regularization
  - Early Stopping
  - L-1 and L-2 Regularization
  - Dropout
  - Data Augmentation
- Depth vs Width

Outline

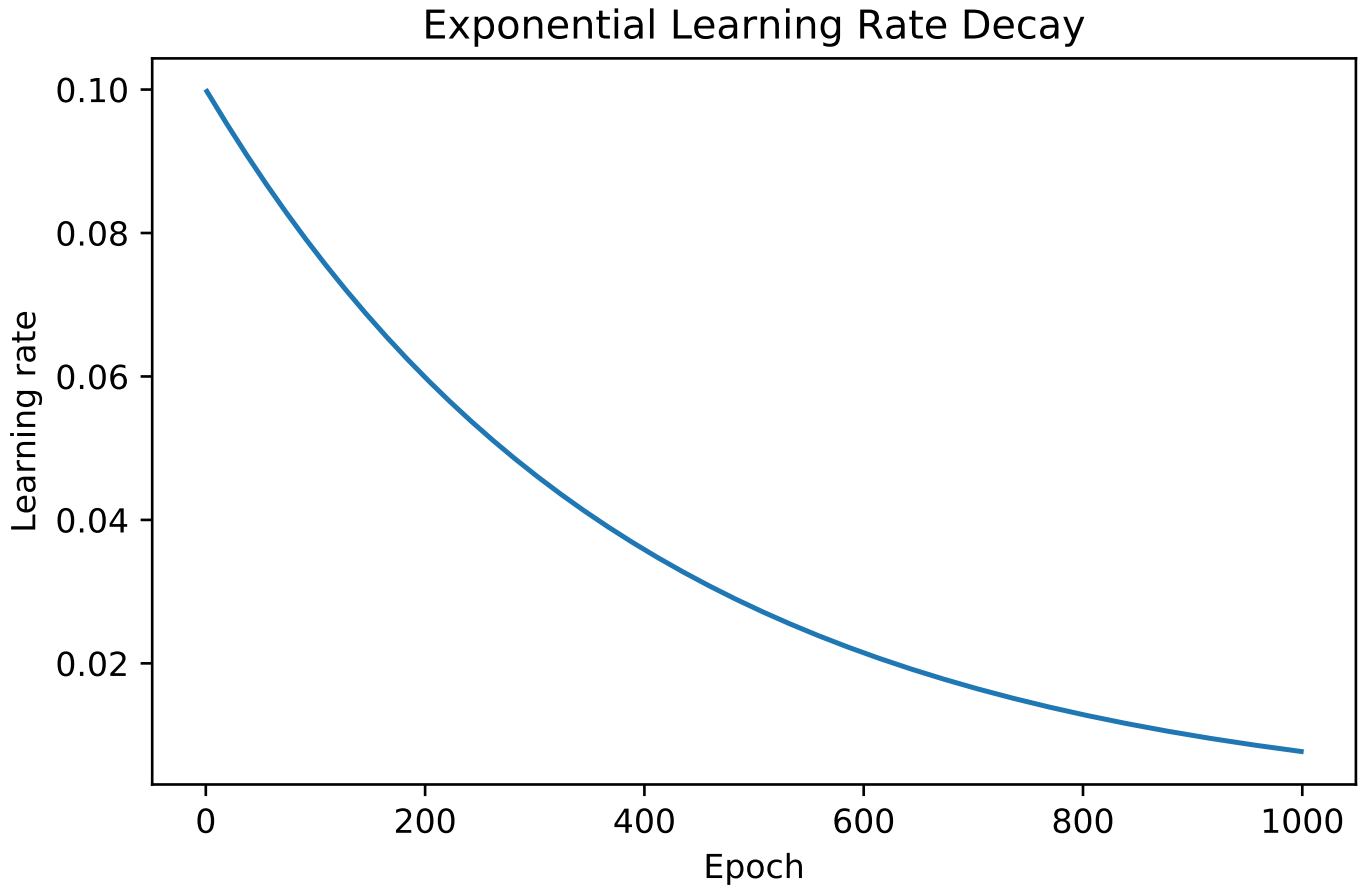
- Embeddings
- Backpropagation
- Faster optimizers
- Global, constant learning rate
  - Momentum
  - Nestorov's Accelerated Gradient
- Global, adaptive learning rate
  - Learning Rate Scheduling
- Local, adaptive learning rate
  - Adaptive Gradient (AdaGrad)
  - RMSprop
  - Adaptive Moment Estimation (ADAM)
- Vanishing/Exploding Gradient
  - Problem statement
  - Solution 1: Weight Initialization
  - Solution 2: Nonsaturating Activation Functions
  - Solution 3: Batch Normalization
  - Solution 4: Gradient Clipping
- Avoiding Overfitting with Regularization
  - Early Stopping
  - L-1 and L-2 Regularization
  - Dropout
  - Data Augmentation
- Depth vs Width

Learning Rate Scheduling

- For Gradient Descent (with or without momentum and Nesterov) and especially Stochastic and Mini-batch Gradient Descent it is recommended that we add a learning schedule.
- Most popular: exponential scheduling: $\eta = \eta_0 r^{t/d}$ where r is decay rate, t is the epoch, d the decay steps, and η_0 the initial learning rate.
- Suppose that we want to multiply the learning rate by a factor $r = 0.95$ every 20 epochs, and start with an initial learning rate of 0.1:

```
lr0 = 0.1
r = 0.95
import numpy as np
t = np.arange(1,1000,1)
lr = lr0*(r**(t/20))
```

Learning Rate Decay



Implementing Learning Rate Decay

Implementing a learning rate schedule with Tensorflow:

```
#initial_learning_rate * decay_rate ^ (currentstep / decay_steps)
initial_learning_rate = 0.1
decay_steps = 10000
decay_rate = 0.95

learning_sched= tf.keras.optimizers.schedules.ExponentialDecay(
initial_learning_rate, decay_steps, decay_rate)

tf.keras.optimizers.SGD(learning_rate=learning_sched)
```

https://www.tensorflow.org/api_docs/python/tf/keras/optimizers/schedules/ExponentialDecay

Outline

- Embeddings
- Backpropagation
- Faster optimizers
- Global, constant learning rate
  - Momentum
  - Nestorov's Accelerated Gradient
- Global, adaptive learning rate
  - Learning Rate Scheduling
- Local, adaptive learning rate
  - Adaptive Gradient (AdaGrad)
  - RMSprop
  - Adaptive Moment Estimation (ADAM)
- Vanishing/Exploding Gradient
  - Problem statement
  - Solution 1: Weight Initialization
  - Solution 2: Nonsaturating Activation Functions
  - Solution 3: Batch Normalization
  - Solution 4: Gradient Clipping
- Avoiding Overfitting with Regularization
  - Early Stopping
  - L-1 and L-2 Regularization
  - Dropout
  - Data Augmentation
- Depth vs Width

Per parameter adaptive learning rate

- Until now we have used a global and equal learning rate for all our parameters (momentum and nesterov) and a global and adaptive learning rate (learning schedule).
- But what if we could speed up or slow down learning by increasing or decreasing this factor, even for each parameter, as the training progresses?
- We could adaptively tune learning and know into which direction to accelerate and decelerate. Several methods that use such adaptive learning rates have been proposed, most notably AdaGrad, RMSprop and ADAM.

Outline

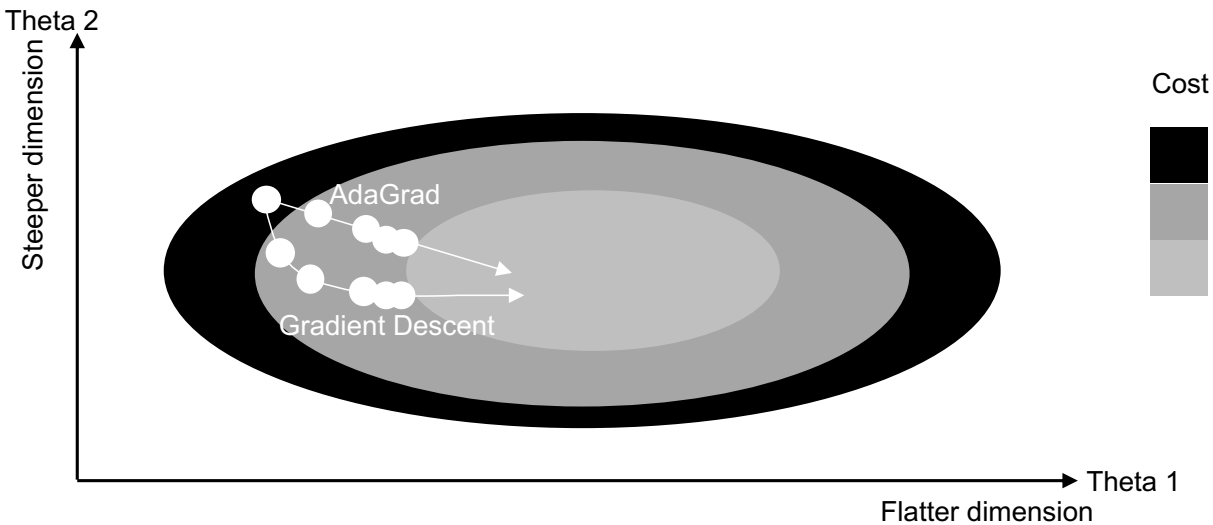
- Embeddings
- Backpropagation
- Faster optimizers
- Global, constant learning rate
  - Momentum
  - Nestorov's Accelerated Gradient
- Global, adaptive learning rate
  - Learning Rate Scheduling
- Local, adaptive learning rate
  - Adaptive Gradient (AdaGrad)
  - RMSprop
  - Adaptive Moment Estimation (ADAM)
- Vanishing/Exploding Gradient
  - Problem statement
  - Solution 1: Weight Initialization
  - Solution 2: Nonsaturating Activation Functions
  - Solution 3: Batch Normalization
  - Solution 4: Gradient Clipping
- Avoiding Overfitting with Regularization
  - Early Stopping
  - L-1 and L-2 Regularization
  - Dropout
  - Data Augmentation
- Depth vs Width

AdaGrad

- GD is guaranteed to reach the minimum loss given convexity as long as we reduce the learning rate as epochs roll on.
- For infrequently occurring features (i.e., they are zero a lot) we may get into a situation where we have reduced the learning rate too quickly. We can't just slow down the reduction in learning rate, because the frequently occurring features will then be jumping around their minima.
- We could keep a counter of how many times a feature was non-zero and divide the learning rate by this number. The learning rate for infrequently occurring features will therefore be higher than the learning rate for frequently occurring features.
- This fails whenever we do not quite have sparsity but rather just data where the gradients are often very small and only rarely large. After all, it is unclear where one would draw the line between something that qualifies as an observed feature or not (if features are small values; nearly zero; the updates will be small too).
- The solution is to take into account the sum of the historical gradients when we divide the learning rate.

AdaGrad

Let's revisit the elongated bowl problem. GD quickly moves down the steepest slope, and then slowly moves down to the bottom. AdaGrad scales down the gradient vector along the steepest dimensions to point more toward the global minimum.



AdaGrad

$$g_t = g_{t-1} + (\nabla_{\theta_t} J(\theta_t))^2$$
$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{diag(g_t) + \epsilon}} \nabla_{\theta_t} J(\theta_t) \tag{3}$$

- AdaGrad adapts the learning rate to the parameters by dividing the learning rate by the square root of the sum of squares of historic gradients. In the denominator
  - the squares are taken to abstract away the sign of the gradients, and the root is taken to undo the effect of squaring and,
  - the sum of the previous gradients is taken so that smaller updates (smaller learning rates) will be used for parameters associated with frequently occurring features, and vice versa for parameters associated with infrequently occurring features.
- g_t is a column vector of length equal to the number of parameters, where an element is the sum of squares of the historic gradients up to iteration t for a parameter.
- $diag(g_t) \in R^{d \times d}$ is a diagonal matrix (to matrix multiply with the gradient vector) where each diagonal element is the sum of the squares of the gradients w.r.t. θ_i up to iteration t and ϵ (say 10^{-8}) is a smoothing term to avoid division by zero

AdaGrad

Summary:

- The learning rate is updated automatically at each time step t
- The learning rate is different per parameter

Pros:

- Speeds up training
- Less need to manually tune learning rate

Cons:

- For each parameter a sum is computed of the squared historic gradients and stored in g_t. Since each element is positive, the sum keeps growing during training, eventually shrinking the learning rate to an infinitesimally small number, and thereby making weight updates impossible.

Tensorflow Implementation of AdaGrad

- AdaGrad often performs well for simple problems but unfortunately it often stops too early when training neural networks. The learning rate gets scaled down so much that the algorithm ends up stopping entirely before reaching the global optimum.

```
tf.keras.optimizers.Adagrad(
learning_rate=0.001, initial_accumulator_value=0.1,
epsilon=1e-07)
```

ORIGINAL PAPER:
http://www.jmlr.org/papers/volume12/duchi11a/duchi11a.pdf

Outline

- Embeddings
- Backpropagation
- Faster optimizers
- Global, constant learning rate
  - Momentum
  - Nestorov's Accelerated Gradient
- Global, adaptive learning rate
  - Learning Rate Scheduling
- Local, adaptive learning rate
  - Adaptive Gradient (AdaGrad)
  - RMSprop
  - Adaptive Moment Estimation (ADAM)
- Vanishing/Exploding Gradient
  - Problem statement
  - Solution 1: Weight Initialization
  - Solution 2: Nonsaturating Activation Functions
  - Solution 3: Batch Normalization
  - Solution 4: Gradient Clipping
- Avoiding Overfitting with Regularization
  - Early Stopping
  - L-1 and L-2 Regularization
  - Dropout
  - Data Augmentation
- Depth vs Width

RMSprop: Root Mean Square Prop

- A disadvantage of AdaGrad is that the g_t part of the update is monotonically increasing. This can pose problems because the learning rate can steadily decrease to the point where it stops the learning altogether.
- RMSprop solves this problem by dividing the learning rate by a running average of squared gradients. For example, if $\gamma = 0.9$ then the first gradient will be 0.9*g_1 in iteration 1, 0.9*(0.9*g_1) in iteration 2, 0.9*(0.9*(0.9*g_1)) in iteration 3, ...
- Here γ is called the decay rate with a typical value of 0.999.
- RMSprop update:

$$
\begin{aligned}
g_t &= \gamma g_{t-1} + (1-\gamma)(\nabla_{\theta_t} J(\theta_t))^2 \\
\theta_{t+1} &= \theta_t - \frac{\eta}{\sqrt{diag(g_t) + \epsilon}} \nabla_{\theta_t} J(\theta_t)
\end{aligned}
\tag{4}
$$

Tensorflow Implementation of RMSprop

Except on very simple problems, this optimizer almost always performs much better than AdaGrad. In fact, it was the preferred optimization algorithm of many researchers until Adam optimization came around. Note: *rho* below is γ in Equation 4.

```
#rho is discounting factor
tf.keras.optimizers.RMSprop(
learning_rate=0.001, rho=0.9, momentum=0.0, epsilon=1e-07)
```

Next, and finally, let's look at ADAM, which is a combination of momentum and RMSprop.

Outline

- Embeddings
- Backpropagation
- Faster optimizers
- Global, constant learning rate
  - Momentum
  - Nestorov's Accelerated Gradient
- Global, adaptive learning rate
  - Learning Rate Scheduling
- Local, adaptive learning rate
  - Adaptive Gradient (AdaGrad)
  - RMSprop
  - Adaptive Moment Estimation (ADAM)
- Vanishing/Exploding Gradient
  - Problem statement
  - Solution 1: Weight Initialization
  - Solution 2: Nonsaturating Activation Functions
  - Solution 3: Batch Normalization
  - Solution 4: Gradient Clipping
- Avoiding Overfitting with Regularization
  - Early Stopping
  - L-1 and L-2 Regularization
  - Dropout
  - Data Augmentation
- Depth vs Width

Adaptive Moment Estimation (ADAM)

- Adam is the state-of-the-art first order optimization algorithm. It is an improvement upon RMSprop by adding momentum to the update rule, combining the best of both momentum and adaptive learning.
- ADAM stores the running average of past squared gradients g_t^2 like RMSprop, and the running average of past gradients v_t like momentum

$$\begin{aligned} g_t &= \gamma_1 g_{t-1} + (1-\gamma_1)(\nabla_{\theta_t} J(\theta_t))^2 \\ v_t &= \gamma_2 v_{t-1} + (1-\gamma_2)\nabla_{\theta_t} J(\theta_t) \end{aligned} \tag{5}$$

The moving average v_t is the first raw (around 0) moment (the mean) of the gradients and the moving average g_t is the second raw moment (the variance) of the gradients. The hyper-parameters γ_1 (default: 0.9) and γ_2 (default: 0.999) are decay rates controlling the weighted averages. t represents the iteration number starting at 1.

Adaptive Moment Estimation (ADAM)

- g_t and v_t are initialized as 0-vectors ($g_1 = 0$ and $v_t = 0$). For this reason, they are biased towards 0 in the beginning of training. We correct for this with the following two equations.
- Compute bias-corrected first and second moment estimates:

$$g_t = \frac{g_t}{1 - \gamma_1^t} \tag{6}$$

$$v_t = \frac{v_t}{1 - \gamma_2^t} \tag{7}$$

  In the beginning of training, these corrections will increase g_t and v_t (i.e., move away from 0). As t increases the denominator will approach 1 and there will be no more correction. In other words, correction only happens in the beginning of training.
- Update rule: $\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{diag(g_t) + \epsilon}} v_t$

Tensorflow Implementation of Adam

Pseudo code in Python:

```
for t in range(num_iterations):
    d = compute the gradient gradient(x, y)
    v = gamma_1 * v + (1 - gamma_1) * d #Momentum
    g = gamma_2 * g + (1 - gamma_2) * np.power(d, 2) #RMSprop
    v = v / (1 - np.power(gamma_1, t)) #Bias Correction
    g = g / (1 - np.power(gamma_2, t)) #Bias Correction
    w = w - step_size * v / np.sqrt(g + epsilon)
```

In Tensorflow:

```
tf.keras.optimizers.Adam(
learning_rate=0.001, beta_1=0.9, beta_2=0.999, epsilon=1e-07)
```

ORIGINAL PAPER: https://arxiv.org/pdf/1412.6980.pdf

Outline

- Embeddings
- Backpropagation
- Faster optimizers
- Global, constant learning rate
  - Momentum
  - Nestorov's Accelerated Gradient
- Global, adaptive learning rate
  - Learning Rate Scheduling
- Local, adaptive learning rate
  - Adaptive Gradient (AdaGrad)
  - RMSprop
  - Adaptive Moment Estimation (ADAM)
- Vanishing/Exploding Gradient
  - Problem statement
  - Solution 1: Weight Initialization
  - Solution 2: Nonsaturating Activation Functions
  - Solution 3: Batch Normalization
  - Solution 4: Gradient Clipping
- Avoiding Overfitting with Regularization
  - Early Stopping
  - L-1 and L-2 Regularization
  - Dropout
  - Data Augmentation
- Depth vs Width

Outline

- Embeddings
- Backpropagation
- Faster optimizers
- Global, constant learning rate
  - Momentum
  - Nestorov's Accelerated Gradient
- Global, adaptive learning rate
  - Learning Rate Scheduling
- Local, adaptive learning rate
  - Adaptive Gradient (AdaGrad)
  - RMSprop
  - Adaptive Moment Estimation (ADAM)
- Vanishing/Exploding Gradient
  - Problem statement
  - Solution 1: Weight Initialization
  - Solution 2: Nonsaturating Activation Functions
  - Solution 3: Batch Normalization
  - Solution 4: Gradient Clipping
- Avoiding Overfitting with Regularization
  - Early Stopping
  - L-1 and L-2 Regularization
  - Dropout
  - Data Augmentation
- Depth vs Width

Vanishing/Exploding Gradient

Preliminaries:

- Assume that we standardized our inputs. This means that they have mean $\mu = 0$, standard deviation $\sigma = 1$, and variance $\sigma^2 = 1^2$.
- Assume that we initialized our weights w from a standard normal distribution (i.e., $\mu = 0$, $\sigma = 1$, and $\sigma^2 = 1^2$).
- Remember that the Variance Product Law in this special case states that for a given w_i and X_i the variance of their product is the product of their variances: $Var(X_i w_i) = 1 * 1 = 1$ when w and X are uncorrelated. If they are correlated we need to add something, but this is immaterial for our discussion.
- Remember that the Variance Sum Law states that the variance σ^2 of the sum equals the sum of the variances. For example, the variance of $z_{h1} = Var(\sum_{i=1}^{m} X_i w_i) = \sum_{i=1}^{m} Var(X_i w_i)$ for m inputs, and w and X column vectors, when the ${w_i}^T X_i$ are uncorrelated. If they are correlated we need to add the covariance between the ${w_i}^T X_i$. The latter is immaterial for our discussion.

Vanishing/Exploding Gradient

Taking into account the preliminaries from the previous slide:

- If there are 200 inputs to a neuron, then $\sigma^2 = 200$ and $\sigma = \sqrt{200} = 14.14$.
- If we have a sigmoid or tanh activation function, and the unactivated weighted sum (z) has a value at $+1$ standard deviation (14.14) from the mean value then the activated value is very close to 1, say 0.9999993.
- Remember that when we backpropagate, to compute the partial derivative we use the chain rule. Imagine a one hidden layer network with two hidden nodes, and one output node. To compute the partial derivative of a weight that is positioned to the left of the hidden layer, one needs to compute the partial derivative of the activated value wrt the weighted sum (in red):

$$\frac{\partial E}{\partial w_1} = \frac{\partial E}{\partial o}\frac{\partial o}{\partial h_1}\frac{\partial h_1}{\partial z_{h_1}}\frac{\partial z_{h_1}}{\partial w_1} \tag{8}$$

Vanishing/Exploding Gradients Problem

When using the sigmoid activation function:

$$\frac{\partial h_1}{\partial z_{h_1}} = sigmoid(z_{h_1})(1 - sigmoid(z_{h_1})) \tag{9}$$

So in our example with $z_{h_1} = 14.14$ (or it could be $z_{h_1} =$ -14.14) we have $\frac{\partial h_1}{\partial z_{h_1}} = 7.228953e - 07$.
When using the tanh activation function:

$$\frac{\partial h_1}{\partial z_{h_1}} = \left(1 - (tanh(x))^2\right) \tag{10}$$

So in our example with $z_{h_1} = 14.14$ (or it could be $z_{h_1} =$ -14.14) we have $\frac{\partial h_1}{\partial z_{h_1}} = 2.090328e - 12$.

Vanishing/Exploding Gradients Problem

To put things in perspective; if we are using the sigmoid function, and z $= 2$, then h will be approximately $1/10$. If z $= 10$ (a mere fivefold increase), then h will be approximately $1/22000$. That means that in the latter case we are learning 2200 times slower.

Vanishing/Exploding Gradients Problem

What are the implications for deep networks?

- Forward pass: If we assume a funnel shaped network (subsequent layers each have less neurons) then more shallow layers will saturate sooner.
- Backward pass: weights going into more shallow layers receive increasingly small updates with every layer as we keep on having to multiply by $\frac{\partial h}{\partial z_h}$.

This is what is called the vanishing gradient problem. It vanishes as we do our backward pass towards more shallow layers, and the reason is that neurons in more shallow layers have increasingly saturated outputs (where the slope is close to zero). The reason why they have increasingly saturated outputs is because more elements need to be added, each having a variance, and the variance of the sum is the sum of the variances. More variance means that we are working in the limits of the tanh and sigmoid functions.

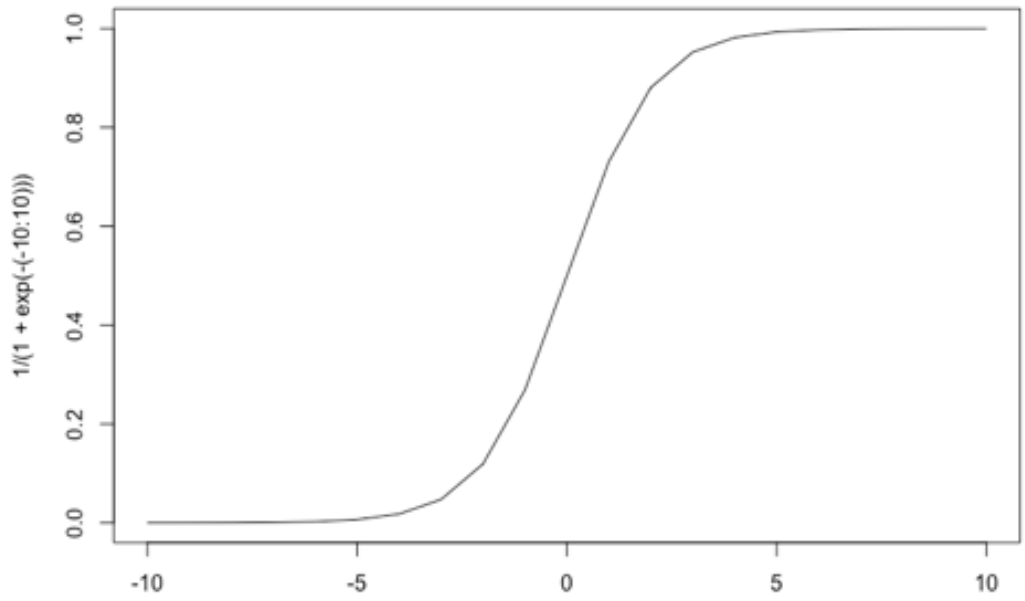
Outline

- Embeddings
- Backpropagation
- Faster optimizers
- Global, constant learning rate
  - Momentum
  - Nestorov's Accelerated Gradient
- Global, adaptive learning rate
  - Learning Rate Scheduling
- Local, adaptive learning rate
  - Adaptive Gradient (AdaGrad)
  - RMSprop
  - Adaptive Moment Estimation (ADAM)
- Vanishing/Exploding Gradient
  - Problem statement
  - Solution 1: Weight Initialization
  - Solution 2: Nonsaturating Activation Functions
  - Solution 3: Batch Normalization
  - Solution 4: Gradient Clipping
- Avoiding Overfitting with Regularization
  - Early Stopping
  - L-1 and L-2 Regularization
  - Dropout
  - Data Augmentation
- Depth vs Width

Solution 1: Weight Initialization

- The goal is thus to make sure that the weighted sum remains inside a feasible range of, say, [-4,4] so that the neurons do not start out learning in a saturated state. A natural solution is therefore to change how weights are initialized, by taking into account the number of incoming weights of a layer.
- For example, a popular weight initialization, called the **normalized initialization**, is $U(\frac{-1}{\sqrt{m}}, \frac{1}{\sqrt{m}})$, with $U[a, b]$ the Uniform distribution in the interval (a,b), and m the size of the previous layer (i.e., number of incoming weights). Biases are set to 0. We have the sqrt in the denominator, because - as the derivation of the variance immediately below shows - this cancels out m from the variance. Using the normalized initialization, the uniform distribution has
  - mean $= \frac{1}{2}(a + b) = 0$
  - variance $= mVar[W] = m\frac{1}{12}(b - a)^2 = m\frac{1}{12}(\frac{1}{\sqrt{m}} - \frac{-1}{\sqrt{m}})^2 = m\frac{1}{12}(\frac{2}{\sqrt{m}})^2 = m\frac{1}{12}(\frac{4}{m}) = \frac{1}{3}$
- In other words: neurons in a given layer are initialized from a uniform distribution that is proportional to how many weights are going to the layer. One is now much less likely to venture outside the [-4,4] range, and saturate the neuron at the start of learning, as for 200 inputs, the $\sigma = \sqrt{1/3}$ as opposed to $\sqrt{200}$ if we would use the normal distribution.

Vanishing/Exploding Gradients Problem

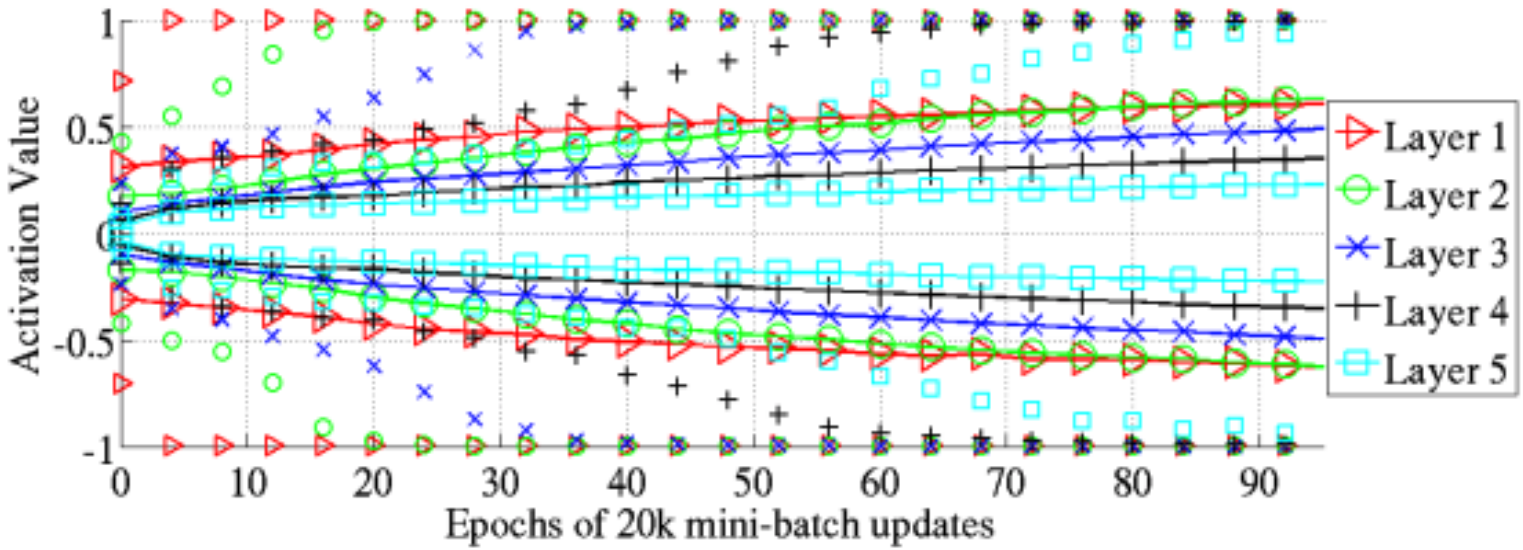


Low variance of z means we are operating in this region

High variance of z means we are operating in this region (extremes are included so higher chance of saturation)

Vanishing/Exploding Gradients Problem

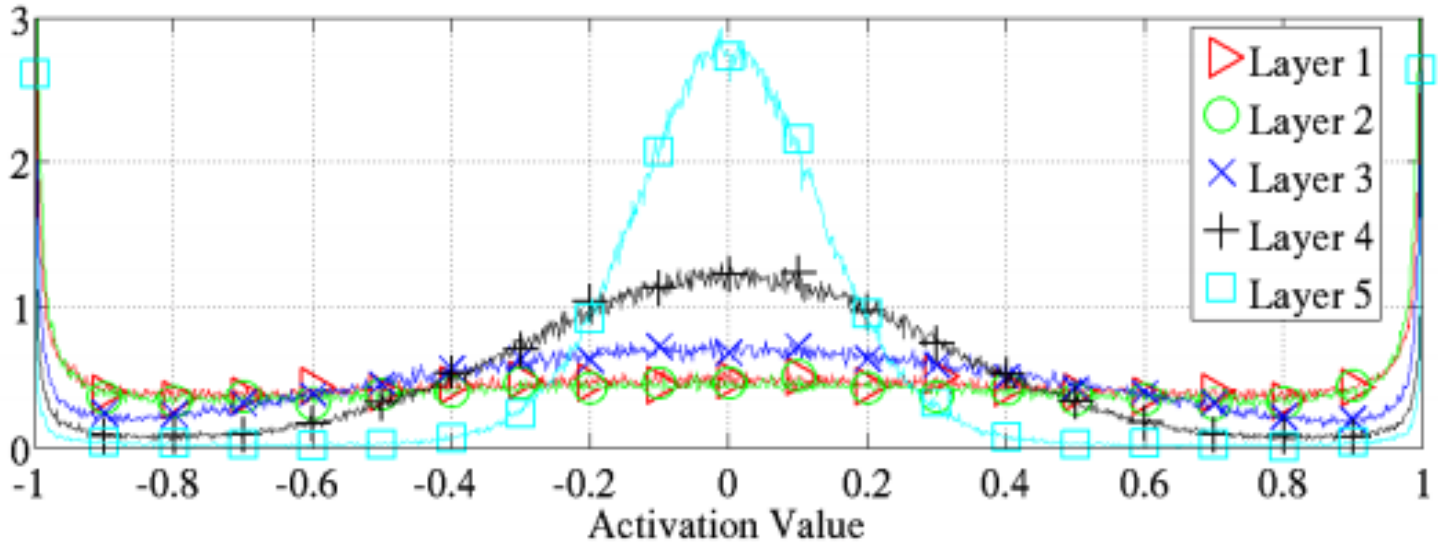
While this has worked in many cases, the problem still persists in many other. For example, using $U(\frac{-1}{\sqrt{m}}, \frac{1}{\sqrt{m}})$: 98 percentiles (markers alone) and standard deviation (solid lines with markers) of the distribution of the activation values for the hyperbolic tangent networks in the course of learning.



More shallow layers saturate sooner.
http://proceedings.mlr.press/v9/glorot10a/glorot10a.pdf

Vanishing/Exploding Gradients Problem

Activation values normalized histogram at the end of learning, averaged across units of the same layer and across 300 test examples. Activation function is hyperbolic tangent, we see important saturation of the lower layers.



More shallow layers saturate more (because they saturate sooner).
http://proceedings.mlr.press/v9/glorot10a/glorot10a.pdf

Weight Initialization

- A breakthrough in solving this issue was presented by Glorot and Bengio (http://proceedings.mlr.press/v9/glorot10a/glorot10a.pdf)
- They posited that, to make sure the signal does not die out, or explodes we need the variance of the outputs of each layer to be equal to the variance of its inputs, and we also need the gradients to have equal variance before and after flowing through a layer in the reverse direction. Formally, with i denoting a layer:
- From a forward pass perspective, the information would flow well if for any h and

$$\forall i, Var(h^i) = Var(h^{i+1}) \tag{11}$$

- From a backward pass perspective, the information would flow well if for any z and

$$\forall i, Var(\frac{\partial E}{\partial z^i}) = Var(\frac{\partial E}{\partial z^{i+1}}) \tag{12}$$

Weight Initialization

These conditions can be rewritten as

$$\forall i, m_i Var(W^i) = 1 \tag{13}$$

$$\forall i, m_{i+1} Var(W^i) = 1 \tag{14}$$

meaning that the variance of the sum of the incoming weights of a given layer and the variance of the sum of the incoming weights for the next layer (i.e., the outgoing weights of the previous layer) equal 1. A compromise between these constraints is the average:

$$\begin{aligned} \forall i, Var(W^i) = \sigma^2 &= \frac{2}{m_i + m_{i+1}} \\ &= \frac{2}{m_{in} + m_{out}} \\ \sigma &= \sqrt{\frac{2}{m_{in} + m_{out}}} \end{aligned} \tag{15}$$

Xavier (Glorot) Initialization

So, the idea is to randomly initialize weights from a Gaussian Distribution with mean 0 and standard deviation:

$$\sigma = \sqrt{\frac{2}{m_{in} + m_{out}}}$$

or from a Uniform distribution between $-r$ and r with,

$$r = \sqrt{\frac{6}{m_{in} + m_{out}}}$$

If $m_{in} = m_{out}$ then we can just do $\sqrt{1/m_{in}}$ and $\sqrt{3/m_{in}}$. The 6 in the numerator of the last equation comes from the fact that the variance of the uniform distribution is $1/3$ (see slide 46), and since we want to keep the variance equal to 1 on average for both incoming and outgoing weights (i.e., 2 divided by the sum of the number of incoming and outgoing weights). These two equations work for the tanh because the value of the derivative of the tanh at the central location is 1.

Xavier (Glorot) Initialization

For the sigmoid function we need a correction factor. The reason is that the derivative at the central location is $1/4$.

$$\sigma = 4\sqrt{\frac{2}{m_{in} + m_{out}}}$$

or from a Uniform distribution between $-r$ and r with,

$$r = 4\sqrt{\frac{6}{m_{in} + m_{out}}}$$

Kaiming He Initialization

For RELU we need yet another correction factor, and this is named after Kaiming He.

$$\sigma = \sqrt{2}\sqrt{\frac{2}{m_{in} + m_{out}}}$$

or from a Uniform distribution between $-r$ and r with,

$$r = \sqrt{2}\sqrt{\frac{6}{m_{in} + m_{out}}}$$

We need to divide m_{in} and m_{out} by 2 inside the square root because RELU reduces the variance (because it clips everything below 0 to 0). Variance closer to 0 leads to activations closer to 0 and therefore we want to penalize less the number of weights. Since a RELU is 0 for half of its input, we need to double the size of weight variance to keep the signal's variance constant.

He Initialization

ORIGINAL PAPER: Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification by Kaiming He, Xiangyu Zhang, Shaoqing Ren and Jian Sun - https://bit.ly/2BQNZXy

Summary Weight Initialization and Tensorflow Implementation

Activation Function	**Uniform Distrib. [-r, +r]**	**Normal Distrib.**
Tanh (Glorot)	$r = \sqrt{\frac{6}{m_{in}+m_{out}}}$	$\sigma = \sqrt{\frac{2}{m_{in}+m_{out}}}$
Sigmoid (Glorot)	$r = 4\sqrt{\frac{6}{m_{in}+m_{out}}}$	$\sigma = 4\sqrt{\frac{2}{m_{in}+m_{out}}}$
ReLU (and variants) (He)	$r = \sqrt{2}\sqrt{\frac{6}{m_{in}+m_{out}}}$	$\sigma = \sqrt{2}\sqrt{\frac{2}{m_{in}+m_{out}}}$

TensorFlow Implementation

```
import tensorflow as tf

#Tanh
#################################
#default is glorot_uniform with uniform distribution
#uniform
keras.layers.Dense(2, activation= 'tanh', kernel_initializer = 'glorot_uniform')

#normal
keras.layers.Dense(2, activation= 'tanh', kernel_initializer = 'glorot_normal')

#Sigmoid
#################################
#uniform
glorot_sigm_init = keras.initializers.VarianceScaling(scale = 16., mode = 'fan_avg',
distribution = 'uniform')

keras.layers.Dense(2, activation= 'sigmoid', kernel_initializer = glorot_sigm_init)

#normal
glorot_sigm_init = keras.initializers.VarianceScaling(scale = 16., mode = 'fan_avg',
distribution = 'untruncated_normal')

keras.layers.Dense(2, activation= 'sigmoid', kernel_initializer = glorot_sigm_init)
```

https://www.tensorflow.org/api_docs/python/tf/keras/initializers/VarianceScaling

TensorFlow Implementation

```
#ReLu and Variants
#################################
#He (based on incoming weights only)
#normal
keras.layers.Dense(2, activation= 'relu', kernel_initializer = 'he_normal')

#uniform
keras.layers.Dense(2, activation= 'relu', kernel_initializer = 'he_uniform')

#He (based on average of incoming and outgoing)
#uniform
he_avg_init = keras.initializers.VarianceScaling(scale = 2., mode = 'fan_avg',
distribution = 'uniform')
keras.layers.Dense(2, activation= 'relu', kernel_initializer = he_avg_init)

#normal
he_avg_init = keras.initializers.VarianceScaling(scale = 2., mode = 'fan_avg',
distribution = 'untruncated_normal')
keras.layers.Dense(2, activation= 'relu', kernel_initializer = he_avg_init)
```

How well does it work?

top panel: $U(\frac{-1}{\sqrt{m}}, \frac{1}{\sqrt{m}})$: more shallow layers have smaller gradients

bottom panel: $U(-\sqrt{\frac{6}{m_{in}+m_{out}}}, \sqrt{\frac{6}{m_{in}+m_{out}}})$

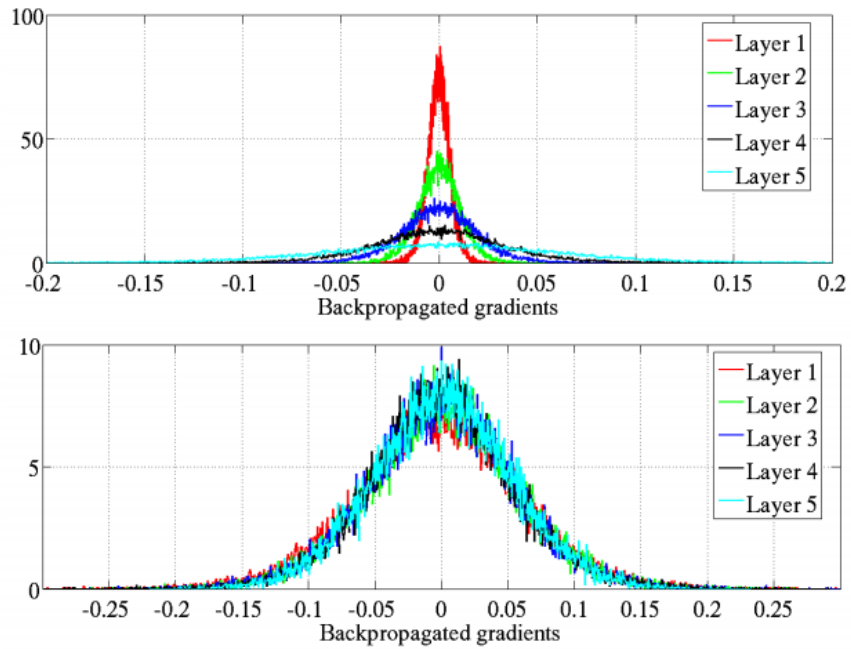


Figure 7: *Back-propagated gradients normalized histograms with hyperbolic tangent activation, with standard (top) vs normalized (bottom) initialization. Top: 0-peak decreases for higher layers.*

Outline

- Embeddings
- Backpropagation
- Faster optimizers
- Global, constant learning rate
  - Momentum
  - Nestorov's Accelerated Gradient
- Global, adaptive learning rate
  - Learning Rate Scheduling
- Local, adaptive learning rate
  - Adaptive Gradient (AdaGrad)
  - RMSprop
  - Adaptive Moment Estimation (ADAM)
- Vanishing/Exploding Gradient
  - Problem statement
  - Solution 1: Weight Initialization
  - Solution 2: Nonsaturating Activation Functions
  - Solution 3: Batch Normalization
  - Solution 4: Gradient Clipping
- Avoiding Overfitting with Regularization
  - Early Stopping
  - L-1 and L-2 Regularization
  - Dropout
  - Data Augmentation
- Depth vs Width

Solution 2: Nonsaturating Activation Functions

- Vanishing/Exploding Gradients problems are in part due to a poor choice of activation function.
- We went through all the activation functions in a previous class and we already know that sigmoid function and hyperbolic tangent function are saturating quickly when the input becomes large. The function saturates, at 0 or 1 for sigmoid function, and -1 and 1 for hyperbolic tangent function, with derivative 0 or extremely close to 0.
- However, other activation functions, the ReLU function and its variants, behave much better in deep neural networks. They do not saturate for positive values and also, they are quite fast to compute. In our eperience, ELU $>$ PRELU $>$ RELU $>$ tanh $>$ sigmoid.

Outline

- Embeddings
- Backpropagation
- Faster optimizers
- Global, constant learning rate
  - Momentum
  - Nestorov's Accelerated Gradient
- Global, adaptive learning rate
  - Learning Rate Scheduling
- Local, adaptive learning rate
  - Adaptive Gradient (AdaGrad)
  - RMSprop
  - Adaptive Moment Estimation (ADAM)
- Vanishing/Exploding Gradient
  - Problem statement
  - Solution 1: Weight Initialization
  - Solution 2: Nonsaturating Activation Functions
  - Solution 3: Batch Normalization
  - Solution 4: Gradient Clipping
- Avoiding Overfitting with Regularization
  - Early Stopping
  - L-1 and L-2 Regularization
  - Dropout
  - Data Augmentation
- Depth vs Width

Solution 3: Batch Normalization

- He initialization with ELU (or variants of RELU) can help us avoid problems at the beginning of learning. Batch normalization helps us stay out of trouble during learning.
- Batch Normalization (BN), introduced in 2015 by Ioffe and Szegedy to address the vanishing/exploding gradient problem, is a widely used technique for normalizing input.
- BN accommodates the **Internal Covariate Shift**: the distribution of each layer's input changes during training as the parameters of previous layers change. **This is a problem because a change in the distributions of layers' inputs requires the layers to continuously adapt to the new distribution.**
- BN involves adding a normalization operation right before or after the activation function of each hidden layer. This operation subtracts from each input variable the mini-batch mean μ_B of the input variable and divides by the mini-batch standard deviation σ_B of the input, where B denotes the mini-batch.

Batch Normalization

But batch normalization does more than that:.

- Simply zero-centering and scaling by the standard deviation as explained on the previous slide would change what the layer can represent. For example, if we normalize the inputs of the sigmoid, then we would always be working in the linear domain of the sigmoid (remember that the sigmoid is only non-linear at very low or very high values) and therefore the neural network model would not be able to represent non-linear functions.
- Ioffe and Szegedy's solution is to ensure that **'the transformation inserted in the network can represent the identity transform'**. In other words, the neural network needs to be given the chance to undo the normalization transformation entirely if it determines that this is the optimal thing to do. Most often this is not the optimal thing to do. What is often optimal is to change the transformation slightly.
- The latter is accomplished by applying, to the normalized inputs $\hat{x}$, a subsequent operation $\gamma\hat{x} + \beta$, where γ and β are a sort of standard deviation and mean (also called the scale and shift parameter), but these are learned by the model itself with backpropagation.

Batch Normalization

Details:

- The normalization is applied to each node k independently. The usual output of the node is here denoted by $x^{(k)}$. If we have a mini-batch of size m instances then we have m such outputs for a node k: $\{x_1^{(k)}, x_2^{(k)}, ..., x_m^{(k)}\}$. The Batch Normalized output is then denoted by $\hat{x}^{(k)}$: $\{\hat{x}_1^{(k)}, \hat{x}_2^{(k)}, ..., \hat{x}_m^{(k)}\}$.
- The γ and β are different per node k: $\gamma^{(k)}\hat{x}^{(k)} + \beta^{(k)}$, and are learned with the other usual weights and biases using backpropagation.
- By setting $\gamma^{(k)} = \sqrt{(Var[x^{(k)}]}$ and $\beta^{(k)} = E[x^{(k)}]$ we could recover the original activations (if that were the optimal thing to do).
- If we were to be operating in a batch setting (each training step uses the whole training set) then we would use the whole set to normalize the activations. However, this is impractical when using incremental learning; therefore we use mini-batches to produce estimates of the mean and variance of each activation.

Batch Normalization

Details, continued:

- The distribution of any $x^{(k)}$ has the expected value of 0 and variance of 1 as long as the elements of each mini-batch are sampled from the same distribution.
- Batch normalization happens before or after every layer (more about this in the implementation section). A scale and shift parameter needs to be learned for every node of every layer that has batch normalization.
- The whole process introduces some noise into the network, and that has been shown to regularize the model to some degree. However, regularization is a side effect of BN, rather than the main objective.
- Summary: γ and β are learned with backpropagation, and μ_B and σ_B^2 are computed on the mini-batch B and used to normalize the mini-batch inputs.
- **ORIGINAL PAPER**: https://arxiv.org/pdf/1502.03167.pdf

Batch Normalization - Algorithm

Input: Values of x over a mini-batch: $B = \{x^1, x^2, ..., x^i, ..., x^m\}$, where m denotes the number of instances in the mini-batch.

Parameters to be learned with backprop:

- γ: output scale vector containing one parameter per input
- β: output shift vector containing one parameter per input

Output (for a given instance, x^i): $y^{(i)} = BN_{\gamma,\beta}(x^{(i)})$

$$\mu_B \leftarrow \frac{1}{m}\sum_{i=1}^{m} x^{(i)} \qquad (mini - batch\ mean)$$

$$\sigma_B{}^2 \leftarrow \frac{1}{m}\sum_{i=1}^{m}(x^{(i)} - \mu_B)^2 \qquad (mini - batch\ variance) \tag{16}$$

$$\hat{x}^{(i)} \leftarrow \frac{x^{(i)} - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} \qquad (normalize)$$

$$y^{(i)} \leftarrow \gamma\hat{x}^{(i)} + \beta \equiv BN_{\gamma,\beta}(x^{(i)}) \qquad (scale\ and\ shift) \tag{17}$$

Notes:

- ϵ is a tiny number to avoid division by zero (called smoothing term).
- μ_B is a vector of the mini-batch means, one for each input.
- σ_B is a vector of the mini-batch standard deviations, one for each input.
- $\hat{x}^{(i)}$ is a vector of zero-centered and normalized inputs for instance i.
- $y^{(i)}$ is the output of the BN operation for instance i.

Batch Normalization - Making Predictions

- We now know that Batch Normalization calculates the mean and variance for each mini-batch at training time. But, what to do at testing (prediction) time? It would not be proper if we are to use mean and variance of the testing mini-batch, because it may be skewed. And what do we do if we predict for a single instance?
- What we do is, we calculate the population average of the mean and variances across all the training mini-batches, and at testing time, we use these two values in the normalization. This provides a more accurate value of the mean and variance.
- To calculate the population averages of the mini-batch mean and variances we keep a moving average so that we do not need to store all the mini-batch means and variances. This is explained in the following slide.

Batch Normalization - Making Predictions

The mini-batch mean vector μ_B (one μ_B for every input) and variance vector σ_B^2 (one σ_B^2 for each input) are computed on every mini-batch and used in the exponentially weighted 'moving average':

$$\begin{aligned} \mu_{mov} &= \alpha\mu_{mov} + (1-\alpha)\mu_B \\ \sigma^2_{mov} &= \alpha\sigma^2_{mov} + (1-\alpha)\sigma_B^2 \end{aligned} \tag{18}$$

Here α is the 'momentum' given to the previous moving statistic, around 0.99.

NOTE: In Tensorflow, the moving mean is initialized from zero and moving variance is initialized from one.

Batch Normalization - Making Predictions

Then, at Testing time, using the population mean and variance, we normalize a given instance x^i as follows:

$$\hat{x}^{(i)} = \frac{x^{(i)} - \mu_{mov}}{\sqrt{\sigma^2_{mov} + \epsilon}} \tag{19}$$
$$y^{(i)} = \gamma \hat{x}^{(i)} + \beta$$

where

- ϵ is a tiny number to avoid division by zero (called smoothing term).
- μ_{mov} is a vector of the population means, one for each input.
- σ_B is a vector of the population standard deviations, one for each input.
- $\hat{x}^{(i)}$ is a vector of zero-centered and normalized inputs for instance i.
- $y^{(i)}$ is the output of the BN operation for instance i.

Batch Normalization - Summary

Training

- Normalize activations according to mini-batch statistics.
- Update training population statistics via moving average of mini-batch statistics.

Testing

- Normalize activations according to estimated training population statistics.
- Do not update population statistics according to mini-batch statistics from test data.

Batch Normalization in Tensorflow

Example. Adding a BatchNormalization layer after every hidden layer and after the input layer:

```
import tensorflow as tf


inputs = tf.keras.layers.Input(shape=(2,), name='input')
BN1 = tf.keras.layers.BatchNormalization(name = 'bn1')(inputs)

hidden1 = tf.keras.layers.Dense(units=2, activation="elu", name = 'h1')(BN1)
BN2 = tf.keras.layers.BatchNormalization(name = 'bn2')(hidden1)

hidden2 = tf.keras.layers.Dense(units=2, activation="elu", name= 'h2')(BN2)
BN3 = tf.keras.layers.BatchNormalization(name = 'bn3')(hidden2)

output = tf.keras.layers.Dense(units=1, activation = "linear", name= 'out')(BN3)

#Create model
model = tf.keras.Model(inputs = inputs, outputs = output)

model.summary()
```

The output is displayed on the next slide.

Batch Normalization in Tensorflow

Example. Output:

```
Model: "functional_1"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
input (InputLayer)           [(None, 2)]               0
_________________________________________________________________
bn1 (BatchNormalization)     (None, 2)                 8
_________________________________________________________________
h1 (Dense)                   (None, 2)                 6
_________________________________________________________________
bn2 (BatchNormalization)     (None, 2)                 8
_________________________________________________________________
h2 (Dense)                   (None, 2)                 6
_________________________________________________________________
bn3 (BatchNormalization)     (None, 2)                 8
_________________________________________________________________
out (Dense)                  (None, 1)                 3
=================================================================
Total params: 39
Trainable params: 27
Non-trainable params: 12
_________________________________________________________________
```

TensorFlow adds four parameters per input: γ, β, μ, and σ. For example, the first batch normalization layer adds 8 parameters (four for each of the two inputs). The parameters μ and σ are the moving averages and are not affected by backpropagation, and therefore they are called non-trainable in TensorFlow. The print screen shows that there are 12 non-trainable parameters, which is the sum of the parameters of all the batch normalization layers divided by 2: $(8 + 8 + 8)/2$.

Batch Normalization in Tensorflow

There is debate about whether to add the Batch Normalization after the activation functions like we just did, or before. It is best to try both and see which option works best on your data. Before the activation function:

```
import tensorflow as tf

inputs = tf.keras.layers.Input(shape=(2,), name='input')
BN1 = tf.keras.layers.BatchNormalization(name = 'bn1')(inputs)

hidden1 = tf.keras.layers.Dense(units=2, name = 'h1')(BN1)
BN2 = tf.keras.layers.BatchNormalization(name = 'bn2')(hidden1)
hidden1act = tf.keras.layers.Activation("elu", name = 'h1act')(BN2)

hidden2 = tf.keras.layers.Dense(units=2, name= 'h2')(hidden1act)
BN3 = tf.keras.layers.BatchNormalization(name = 'bn3')(hidden2)
hidden2act = tf.keras.layers.Activation("elu", name = 'h2act')(BN3)

output = tf.keras.layers.Dense(units=1, activation = "linear", name= 'out')(hidden2act)

#Create model
model = tf.keras.Model(inputs = inputs, outputs = output)

model.summary()
```

Note that above the activation is removed from the dense layers, and that we can set use_bias = False since batch normalization has an offset parameter per input.

Batch Normalization Performance

Benefits of batch normalization, summarized:

- Batch Normalization considerably improves deep neural nets by strongly reducing the vanishing gradient problem. It makes the networks much less sensitive to initialization (even saturating functions can be used)
- It regularizes the network helping overfitting prevention
- When you start using batch normalization you will notice that it slows down training and that is because it needs to train extra parameters (Scale and Offset) for each node in a layer that is batch normalized. However, the batch normalization will actually speed up learning because substantially less training steps will be required to reach the same model performance because larger learning rates can be used. Each epoch will be slower but we will have to do less epochs.

Outline

- Embeddings
- Backpropagation
- Faster optimizers
- Global, constant learning rate
  - Momentum
  - Nestorov's Accelerated Gradient
- Global, adaptive learning rate
  - Learning Rate Scheduling
- Local, adaptive learning rate
  - Adaptive Gradient (AdaGrad)
  - RMSprop
  - Adaptive Moment Estimation (ADAM)
- Vanishing/Exploding Gradient
  - Problem statement
  - Solution 1: Weight Initialization
  - Solution 2: Nonsaturating Activation Functions
  - Solution 3: Batch Normalization
  - Solution 4: Gradient Clipping
- Avoiding Overfitting with Regularization
  - Early Stopping
  - L-1 and L-2 Regularization
  - Dropout
  - Data Augmentation
- Depth vs Width

Solution 4: Gradient Clipping

- In contrast to the vanishing gradient problem there is also the exploding gradient problem. We will learn about the conditions that are necessary for this to happen when covering recurrent neural networks. One solution is to clip the gradients to be between a lower and upper bound.
- Choosing the threshold is the hardest part. You should use the biggest value such that you don't have an exploding gradient (whose effects can be NANs or infinite values appearing in your tensors, constant loss /accuracy after a few training steps).
- The code below will clip every component of the gradient vector to a value between -1.0 and 1.0 and is called **clipping by value** of every component separately:

```
optimizer = tf.keras.optimizers.SGD(clipvalue = 1.0)
model.compile(loss = 'mse', optimizer = optimizer)
```

For example a gradient vector with two partial derivatives [0.8, 20.0] will become [0.8, 1.0]. Before the clipping the direction was towards the second axis, and after clipping the direction is more towards the diagonal between the two axes. In other words, clipping by value can change the direction of the gradient vector.

Gradient Clipping

- If we do not want to change the direction of the gradient then we can **clip by norm**. This involves changing all the components of the gradient vector if the L2 norm ($\sqrt{x_1^2 + x_2^2 + ...}$) surpasses a given threshold. The code below will rescale every component of the gradient vector if the L2 norm exceeds 1.0 so that the L2 norm equals 1.0.

```
optimizer = tf.keras.optimizers.SGD(clipnorm = 1.0)
model.compile(loss = 'mse', optimizer = optimizer)
```

For example, a gradient vector with two partial derivatives [0.8, 20.0] will become [0.03996804, 0.99920096].

```
import numpy as np
np.array([0.8,20])/np.sqrt(0.8**2 + 20**2)
```

This preserves the direction of the gradient: it is pointing primarily to the second axis before and after clipping.

Outline

- Embeddings
- Backpropagation
- Faster optimizers
- Global, constant learning rate
  - Momentum
  - Nestorov's Accelerated Gradient
- Global, adaptive learning rate
  - Learning Rate Scheduling
- Local, adaptive learning rate
  - Adaptive Gradient (AdaGrad)
  - RMSprop
  - Adaptive Moment Estimation (ADAM)
- Vanishing/Exploding Gradient
  - Problem statement
  - Solution 1: Weight Initialization
  - Solution 2: Nonsaturating Activation Functions
  - Solution 3: Batch Normalization
  - Solution 4: Gradient Clipping
- Avoiding Overfitting with Regularization
  - Early Stopping
  - L-1 and L-2 Regularization
  - Dropout
  - Data Augmentation
- Depth vs Width

Regularization

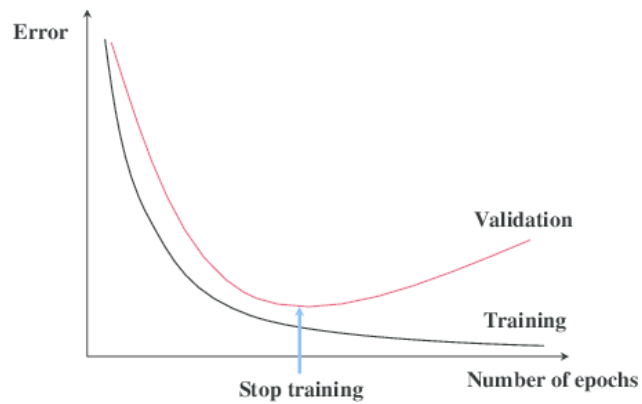
- Deep neural networks typically have tens of thousands of parameters, sometimes even millions.
- With so many parameters, the network has an incredible amount of freedom and can fit a huge variety of complex datasets.
- But this great flexibility also means that it is prone to over-fitting the training set.
- In this section, we will present some of the most popular regularization techniques for neural networks.

Outline

- Embeddings
- Backpropagation
- Faster optimizers
- Global, constant learning rate
  - Momentum
  - Nestorov's Accelerated Gradient
- Global, adaptive learning rate
  - Learning Rate Scheduling
- Local, adaptive learning rate
  - Adaptive Gradient (AdaGrad)
  - RMSprop
  - Adaptive Moment Estimation (ADAM)
- Vanishing/Exploding Gradient
  - Problem statement
  - Solution 1: Weight Initialization
  - Solution 2: Nonsaturating Activation Functions
  - Solution 3: Batch Normalization
  - Solution 4: Gradient Clipping
- Avoiding Overfitting with Regularization
  - Early Stopping
  - L-1 and L-2 Regularization
  - Dropout
  - Data Augmentation
- Depth vs Width

Early Stopping

- Validation error can be used to detect when overfitting starts
- The goal is to stop training once validation error reaches a minimum. This is called **early stopping** because we are stopping training before the training loss has converged. This is not a simple task, as we only know when we have reached a minimum once validation error starts increasing, and the error curve is not always smooth.
- One solution is to use **patience**; the number of epochs after which the validation error has remained above the minimum, before stopping training, and then rolling back the model parameters to the point where the validation error was at a minimum. The patience is often set somewhere between 10 and 20, but it really depends on the dataset and network.



Implementing Early Stopping

- First, let's see how we can save and load models:

```
#model.fit....
#To save a model:
model.save('my_model.keras')
#To load a model:
model = tf.keras.models.load_model('my_model.keras')
```

- This will work with the Functional API, and saves the values of all model parameters and the optimizer including its hyperparameters and any state it may have.
- One typically has a script that trains the model, and then one or more other scripts to load the model and deploy it.

Implementing Early Stopping

- If we have epochs > 1 in our fit call (i.e., we do not have a manual loop for epochs and rows), then we can also save a checkpoint
- This is accomplished with a callback function. A callback function is ran at specified times during the execution of the fit call (here at the end of every epoch, but it can also be after x batches). In addition to running at the end of every epoch it also runs at the beginning and end of training.

```
checkpoint_cb = tf.keras.callbacks.ModelCheckpoint(
        filepath = 'my_model.keras',
        save_freq = 'epoch')
#filepath, if not specifically formatted to save the model
#in a new directory every time then the model will be
#overwritten each time to this filepath.
#(www.tensorflow.org/api_docs/python/tf/keras/callbacks/
# ModelCheckpoint)
#If save_freq = 'epoch' it saves the model after every epoch.
#save_freq can also be an integer, which denotes to save
#the model every save_freq batches.
model.fit(xtrain, ytrain, epochs = 100,
callbacks = [checkpoint_cb])
```

Implementing Early Stopping

- Provided that we are overwriting the model every time, and provided that we have a validation set during training (required for early stopping) then we can instruct TensorFlow to refrain from overwriting the **best** model based on validation performance. In other words TensorFlow will only save (overwrite) the model if it is better than before. This is accomplished with the save_best_only = True setting.
- By default it will monitor the validation loss for this purpose, but one can change this if desired to an arbitrary measure.

```
checkpoint_cb = tf.keras.callbacks.ModelCheckpoint(
        filepath = 'my_model.keras',
        save_freq = 'epoch',
        save_best_only = True)

model.fit(xtrain, ytrain,
                validation_data = (xval, yval),
                epochs = 100,
                callbacks = [checkpoint_cb])

#roll back to the best model after 100 epochs:
model = tf.keras.model.load_model('my_model.keras')
```

Implementing Early Stopping

- With the implementation of early stopping on the previous slide we do not have to worry about training too long and overfitting. However, the model will run for 100 epochs unconditionally, which means we may be spending computational resources unnecessarily.
- We can expand the code on the previous slide to, in addition to saving and not overwriting the best model, have it stop training after a given number of epochs of remaining above the minimum validation loss. This number is called the **patience**.

```
checkpoint_cb = tf.keras.callbacks.ModelCheckpoint(
                filepath = 'my_model.h5',
                save_freq = 'epoch',
                save_best_only = True)
early_stopping_cb = tf.keras.callbacks.EarlyStopping(patience = 10,
        restore_best_weights = True)
model.fit(xtrain, ytrain,
              validation_data = (xval, yval),
              epochs = 100,
              callbacks = [checkpoint_cb, early_stopping_cb])
#We do ****not*** need to run the following line if we are working
#in the same session since restore_best_weights = True in
#callbacks.EarlyStopping does that for us. Of course we would
#need this if we want to use the model in a separate session.
#model = tf.keras.model.load_model('my_model.keras')
```

Implementing Early Stopping

- Unfortunately, all these convenience functions do not work (or are at the least inefficient) when we have a manual epoch and instance loop.
- In the bonus assignment we will implement early stopping when we are in this specific situation.

Outline

- Embeddings
- Backpropagation
- Faster optimizers
- Global, constant learning rate
  - Momentum
  - Nestorov's Accelerated Gradient
- Global, adaptive learning rate
  - Learning Rate Scheduling
- Local, adaptive learning rate
  - Adaptive Gradient (AdaGrad)
  - RMSprop
  - Adaptive Moment Estimation (ADAM)
- Vanishing/Exploding Gradient
  - Problem statement
  - Solution 1: Weight Initialization
  - Solution 2: Nonsaturating Activation Functions
  - Solution 3: Batch Normalization
  - Solution 4: Gradient Clipping
- Avoiding Overfitting with Regularization
  - Early Stopping
  - L-1 and L-2 Regularization
  - Dropout
  - Data Augmentation
- Depth vs Width

L-1 and L-2 Regularization

LASSO, short for Least Absolute Shrinkage and Selection Operator, is a penalty added to the loss, and equal to the sum of the absolute value of all n weights (i.e., the L1 norm):

$$loss(x, y, \theta) + \lambda \sum_{i=1}^{n} \mid \theta_i \mid \tag{20}$$

Ridge (L2), is a penalty added to the loss, and equal to the sum of all n weights squared (i.e., the L2 norm):

$$loss(x, y, \theta) + \frac{\lambda}{2} \sum_{i=1}^{n} \theta_i^2 \tag{21}$$

Elastic Net is a mix of Lasso and Ridge and how much each contributes is controlled by a mixing parameter r:

$$loss(x, y, \theta) + r\lambda \sum_{i=1}^{n} \mid \theta_i \mid + (1 - r)\frac{\lambda}{2} \sum_{i=1}^{n} \theta_i^2 \tag{22}$$

L-1 and L-2 Regularization

Details:

- The Ridge (L2) loss is multiplied by $\frac{1}{2}$ because the derivative of θ^2 is 2θ. This means in derivative space every weight counts as $\lambda\theta$, as opposed to $\lambda 2\theta$ (there is no reason to have the 2 in there).
- Penalties added to the loss are sensitive to the scale of the data. For example, a variable with a larger scale will have a smaller weight, and thus that variable will be penalized less. When using penalties, inputs should always be scaled.
- In L1 (LASSO), weights shrink by a constant amount, whereas in L2 (Ridge) the weights shrink by an amount proportional to the weights. When weights are large, L1 shrinks the weights a lot less than L2. When weights are small, L1 shrinks weights a lot more. For example, if an initial weight is small, 0.1, then L2 will grow by only 0.01 while L1 grows by 0.1. If the initial weight is 3, then L1 will grow by only 3, while L2 will grow by 9. Therefore, L1 is said to be sparser than L2. Small weights (those between -1 and 1) are penalized disproportionately little in L2 compared to weights outside that range, while in L1 they are penalized proportionally. This means that in L1 weights are sparser (they arrive at zero faster) than in L2 (it is much harder to get them to be zero).

L-1 and L-2 Regularization

L1:

```
tf.keras.layers.Dense(10,
                      activation="elu",
                      kernel_initializer="he_normal",
                      kernel_regularizer=tf.keras.regularizers.l1(0.01))
```

L2:

```
tf.keras.layers.Dense(10,
                      activation="elu",
                      kernel_initializer="he_normal",
                      kernel_regularizer=tf.keras.regularizers.l2(0.01))
```

Elastic Net (note the lack of an explicit mixing parameter; we can set different λ values instead) :

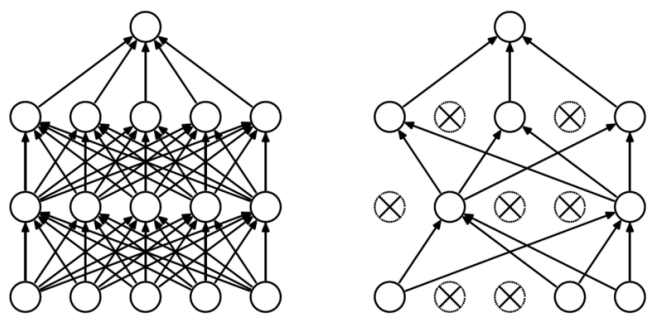
```
tf.keras.layers.Dense(10,
  activation="elu",
  kernel_initializer="he_normal",
  kernel_regularizer=tf.keras.regularizers.l1_l2(l1=0.01,l2=0.01))
```

Outline

- Embeddings
- Backpropagation
- Faster optimizers
- Global, constant learning rate
  - Momentum
  - Nestorov's Accelerated Gradient
- Global, adaptive learning rate
  - Learning Rate Scheduling
- Local, adaptive learning rate
  - Adaptive Gradient (AdaGrad)
  - RMSprop
  - Adaptive Moment Estimation (ADAM)
- Vanishing/Exploding Gradient
  - Problem statement
  - Solution 1: Weight Initialization
  - Solution 2: Nonsaturating Activation Functions
  - Solution 3: Batch Normalization
  - Solution 4: Gradient Clipping
- Avoiding Overfitting with Regularization
  - Early Stopping
  - L-1 and L-2 Regularization
  - Dropout
  - Data Augmentation
- Depth vs Width

Dropout

- Dropout (Srivastava, Hinton, et al, 2014) is very popular because it works so well and is simple to understand.
- At every training step, every neuron (input neurons and hidden neurons, but not output neurons) has a probability p of being temporarily **dropped out**. This means that it will be completely ignored (it outputs 0) during that training step, but it may be active during the next step.
- A typical dropout rate is between 10% and 50%.
- After training, neurons are not dropped anymore.
- Note that dropout slows down convergence, but results in better models.
- **ORIGINAL PAPER**: https://www.cs.toronto.edu/~hinton/absps/JMLRdropout.pdf



(a) Standard Neural Net (b) After applying dropout.

Why does Dropout work?

Explanation 1: ensembling

- A unique neural network is generated at each training step. Since each neuron can be either present or absent, there are a total of 2^n (2 for either present or absent) possible 'thinned' networks where n is the total number of droppable neurons. These networks are not independent since they share many of their weights but they are all different.
- Therefore, while implementing dropout, we are training many different networks, each with different (but shared) parameters. And when it is time to make predictions (dropout is only active at training time), the resulting network roughly represents the combination of all these networks which nearly always improves the performance of machine learning methods.
- Example: If we would train a network with 100,000 training iterations we have 100,000 different networks
- This can be perceived of as an averaging **ensemble** of all these smaller networks.

Why does Dropout work?

Explanation 2: resilience

- Consider a soccer team with 11 players. Some players specialize playing in the front, some in the middle, and some in the back.
- For every game we would flip a coin for each of the players, to decide if they can play or not. This means that if we would do that many times we would have 5 to 6 players in the team every game.
- In the last game the key front player was not allowed to play, so another player had to play that position and while doing so, gathered valuable experience, which they would have never learned if all 11 players were playing. The team has therefore become much more resilient.
- It is unclear if this system would work well in soccer, but it does work really well in neural networks. Neurons cannot co-adapt with neighboring neurons, and they cannot rely excessively on a few input neurons (since any input neuron can also be dropped out). This makes neurons less sensitive to small changes in the inputs, which reduces overfitting.

Important Detail

- Remember that dropout is on during training, and off during predicting
- If neurons are retained with probability 50% during training (the keep probability, or 1 - dropout probability), then every neuron will be connected to twice as many input neurons during prediction as it was on average during training.
- To compensate, during prediction after training, we need to **multiply** all the **incoming weights** of each neuron by 50% (the keep probability), otherwise the signal will be twice as large during prediction as what the network was trained on.
- Alternatively, we can **divide** each neuron's **output** by the keep probability during training. In other words, there will be less weights and therefore we boost the output. TensorFlow does this for us when using tf.keras.layers.Dropout; we do not need to do this manually.
- In sum, during prediction double (if keep probability was 50%) the weights will be active and so we need to either halve the signal by multiplying weights by 50% during prediction, or double the signal during training

Practical Guidelines of Dropout

Do not apply dropout

- right after the last layer, because the network has no ability to correct for dropout in subsequent layers.
- when the network is small relative to the dataset, regularization is usually unnecessary. If the model capacity is already low, lowering it further by adding regularization will hurt performance.
- when training time is limited. If we don't train until convergence (e.g., we use early stopping), dropout may give worse results. Usually dropout hurts performance at the start of training, but results in the final "converged" error being lower. Therefore, if we don't plan to train until convergence, we may not want to use dropout.

Other things to try:

- Increase the dropout rate when the model is overfitting, and decrease it when it is underfitting
- Add dropout only after the last (few) hidden layers in case of underfitting

Tensorflow Implementation of Dropout

- See **dropout.py** for a complete example. Each dropout layer drops some neurons (sets them to 0) and divides the remaining neurons' outputs by the keep probability
- In 2016, Gal and Ghahramani made a modification to dropout during the prediction phase called Monte Carlo Dropout (MC Dropout) that was shown to improve prediction performance. They also made a profound connection to approximate Bayesian Inference and ensembling.
- Their simple suggestion was to turn dropout on during prediction, and deploy the model k times on each record, resulting in k predictions per record. These k predictions can then be averaged (and we can also compute their standard deviation, for example). They showed that this average was a better prediction than making a single prediction on each record with dropout turned off.
- MC Dropout can be accomplished by tricking TensorFlow that it is in training mode, thereby activating the dropout layers. The following line will predict a different value each time for every instance:

```
y_hat = model(X, training = True)
```

BatchNorm and Dropout at the Same Time?

When they are combined, they can lead to a worse performance.
https://arxiv.org/pdf/1801.05134.pdf

Outline

- Embeddings
- Backpropagation
- Faster optimizers
- Global, constant learning rate
  - Momentum
  - Nestorov's Accelerated Gradient
- Global, adaptive learning rate
  - Learning Rate Scheduling
- Local, adaptive learning rate
  - Adaptive Gradient (AdaGrad)
  - RMSprop
  - Adaptive Moment Estimation (ADAM)
- Vanishing/Exploding Gradient
  - Problem statement
  - Solution 1: Weight Initialization
  - Solution 2: Nonsaturating Activation Functions
  - Solution 3: Batch Normalization
  - Solution 4: Gradient Clipping
- Avoiding Overfitting with Regularization
  - Early Stopping
  - L-1 and L-2 Regularization
  - Dropout
  - Data Augmentation
- Depth vs Width

Data Augmentation

- One last regularization technique is data augmentation.
- It consists of generating new training instances from existing ones, artificially boosting the size of the training set.
- This will reduce overfitting. The trick is to generate realistic training instances. Ideally a human should not be able to tell which instances were generated and which ones were not.
- Adding simply white noise will not help because it is not learnable.
- Mostly used in Convolutional Neural Networks (see later).
- Instances can be generated on the fly during training

Outline

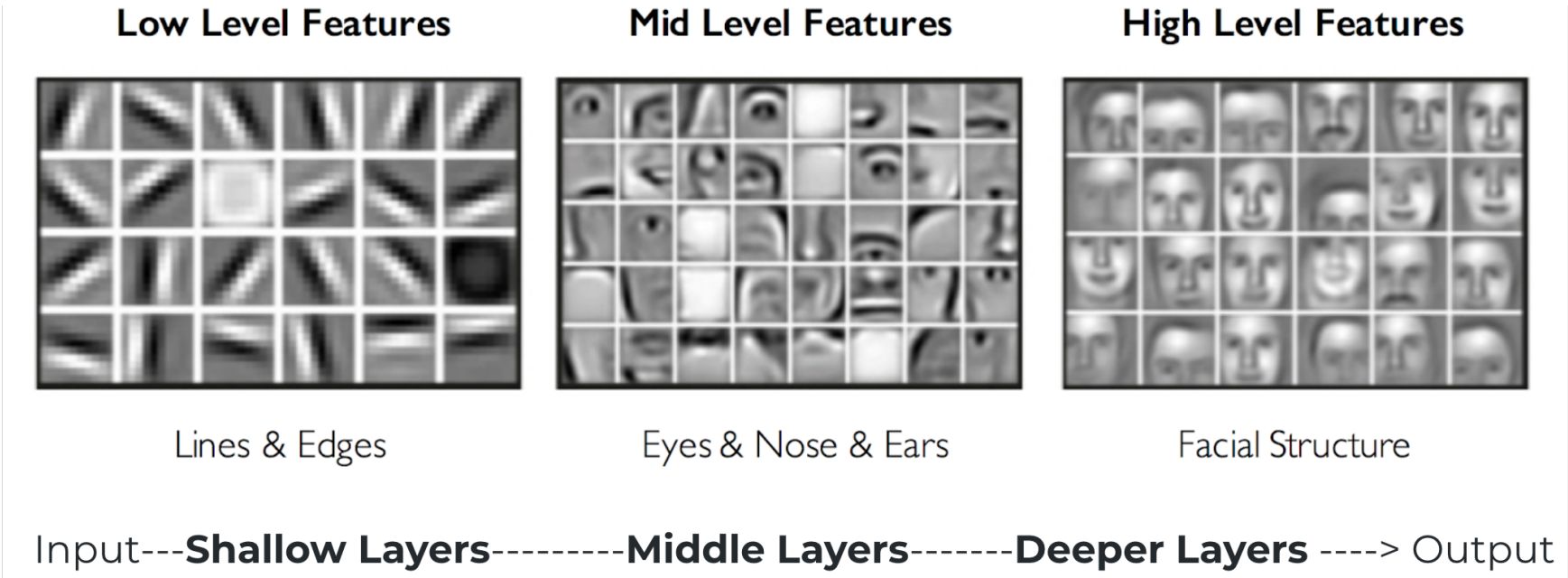
- Embeddings
- Backpropagation
- Faster optimizers
- Global, constant learning rate
  - Momentum
  - Nestorov's Accelerated Gradient
- Global, adaptive learning rate
  - Learning Rate Scheduling
- Local, adaptive learning rate
  - Adaptive Gradient (AdaGrad)
  - RMSprop
  - Adaptive Moment Estimation (ADAM)
- Vanishing/Exploding Gradient
  - Problem statement
  - Solution 1: Weight Initialization
  - Solution 2: Nonsaturating Activation Functions
  - Solution 3: Batch Normalization
  - Solution 4: Gradient Clipping
- Avoiding Overfitting with Regularization
  - Early Stopping
  - L-1 and L-2 Regularization
  - Dropout
  - Data Augmentation
- Depth vs Width

Deep Neural Networks

- In recent years, neural networks have become deeper and deeper, with state-of-the-art networks going from 7 layers (AlexNet) to 1000 layers (Residual Nets) in the space of 4 years.
- However, the number of units in the fully-connected layers, has remained roughly the same and is still relatively small in magnitude, despite the large increase in the number of layers.
- The advantage of multiple layers is that they attempt to model various levels of abstractions in data by using architectures composed of multiple non-linear transformations.
- The idea draws from how animals and humans learn: with simpler concepts earlier in life and higher-level abstractions later, expressed in terms of the previously learned concepts.
- Multiple layers are much better at generalizing because they learn all the intermediate features between the raw data and the high-level classification.

Deep Neural Networks

For example, if we train a deep convolutional neural network to classify images, we will find that the first layer will train itself to recognize very basic things such as lines and edges, the next layer will train itself to recognize collections of shapes such as eyes/noses/ears, and the next layer will learn even higher-order features such as faces.



Wide Neural Networks

Then, how about going wider (i.e., more neurons, less layers)?

- The advantage of wide neural networks is that the model will be very good at memorization. Memorization can be loosely defined as learning the frequent co-occurrence of items or features and exploiting the correlation available in the historical data.
- Widening consistently improves performance across networks of different depth.
- Increasing width helps until the number of parameters becomes too large and stronger regularization is needed.
- Wider networks allow many multiplications to be computed in parallel, while deeper networks use more sequential computations (since the computations depend on the previous layer). Modern GPUs are better suited for parallel computing
- Increasing depth results in better generalization; exploring new feature combinations that have never or rarely occurred in the past.